

JURISPULSE

21-DAY DEVELOPMENT & GIT HISTORY PLAN

A day-relative engineering roadmap for a clean, granular repository history.

21 days	388 commit slots	15-20 commits / day	Day-relative no fixed dates
-------------------	----------------------------	-------------------------------	---------------------------------------

Purpose

Use this document as the single source of truth for a staged project upload. The sequence follows the architecture already documented for JurisPulse: React/TypeScript/Vite/Tailwind on the frontend; FastAPI, SQLAlchemy, PostgreSQL/pgvector, Celery/Redis, document/OCR services, research/retrieval/verification, AI services, infrastructure, testing, and documentation.

Important operating rule

Each commit slot is a small engineering change. Antigravity should map the slot to real files and real changes in the repository. Do not create empty commits, fabricate work, or claim a change that was not actually made. When a slot does not match the current codebase, skip it or replace it with another real change from the same day theme.

Daily routine

1. You announce: "Day X is today."
2. Antigravity reads this PDF from the repository root.
3. It audits the current tree and identifies only the files relevant to Day X.
4. It makes the day's changes in small, reviewable increments.

5. It commits each completed change separately with a concise, normal engineering message.
6. It runs the relevant validation before moving on.
7. It stops after the day's scope; it does not pull work forward from future days unless needed to repair a broken dependency.

Roadmap at a glance

Day	Theme	Commits
Day 01	Repository foundation and baseline	17
Day 02	Frontend toolchain and configuration	16
Day 03	Frontend application shell	17
Day 04	Design system and global styling	15
Day 05	Reusable frontend components	18
Day 06	Authentication frontend	17
Day 07	Cases frontend	18
Day 08	Documents and evidence frontend	20
Day 09	Hearings, timeline, clients and collaboration UI	18
Day 10	Research, analytics, reports and notifications UI	19
Day 11	Backend application foundation	17
Day 12	Database, security and migrations	18
Day 13	Users, organizations, roles and admin backend	19
Day 14	Cases, hearings, timeline and evidence backend	20
Day 15	Documents, OCR, drafting and storage backend	20
Day 16	Research, retrieval, verification and AI services	20
Day 17	Workers, workflows, tasks, notifications and collaboration backend	20
Day 18	API integration, error handling and security hardening	19
Day 19	Testing, quality gates and CI	20
Day 20	Infrastructure, deployment and documentation	20
Day 21	Refactor, polish, final validation and release preparation	20

Commit message style

Prefer short, specific messages such as Add case list page, Normalize database session, Add OCR service, Connect research API, or Refine error handling. The message should describe the actual change, not hype the change or describe the use of AI. Avoid repetitive phrases, fake issue numbers, and filler such as "comprehensive enhancement" unless the repository genuinely contains such a broad change.

Day 01

Repository foundation and baseline

17 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|------------------------------------|---------------------------------------|
| 01 Initialize repository structure | 10 Create scripts workspace |
| 02 Add root .gitignore | 11 Create tests workspace |
| 03 Add .editorconfig | 12 Create infrastructure workspace |
| 04 Add LICENSE file | 13 Create GitHub metadata folder |
| 05 Add root README skeleton | 14 Normalize root configuration files |
| 06 Add PROJECT_STRUCTURE.md | 15 Remove generated build artifacts |
| 07 Create frontend workspace | 16 Remove cache and temporary files |
| 08 Create backend workspace | 17 Record clean baseline |
| 09 Create docs workspace | |

Day 01 is intentionally repository-level. Keep it focused on cleanup, organization, and a clean baseline. Do not start feature work early.

Day 02

Frontend toolchain and configuration

16 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|-------------------------------------|---------------------------------------|
| 01 Add package manifest cleanup | 09 Configure linting |
| 02 Normalize package scripts | 10 Configure formatting |
| 03 Configure TypeScript | 11 Add frontend test configuration |
| 04 Configure Vite | 12 Add public assets structure |
| 05 Configure Tailwind | 13 Normalize entry files |
| 06 Add PostCSS configuration | 14 Add frontend README |
| 07 Add frontend environment example | 15 Validate frontend dependency graph |
| 08 Configure path aliases | 16 Run initial frontend build |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 03

Frontend application shell

17 commit slots - Complete in order unless the current repository state makes a different order safer.

- 01 Add React application entry
- 02 Add root App component
- 03 Add application router
- 04 Add route configuration
- 05 Add provider composition
- 06 Add application layout
- 07 Add authenticated layout
- 08 Add public layout
- 09 Add navigation shell

- 10 Add sidebar structure
- 11 Add top bar structure
- 12 Add page container
- 13 Add route fallback
- 14 Add loading boundary
- 15 Add global error boundary
- 16 Validate application routing
- 17 Run frontend typecheck

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 04

Design system and global styling

15 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|--|----------------------------------|
| 01 Add design tokens | 09 Normalize global CSS |
| 02 Add typography tokens | 10 Add form styling primitives |
| 03 Add spacing tokens | 11 Add surface primitives |
| 04 Add radius tokens | 12 Add responsive breakpoints |
| 05 Add shadow tokens | 13 Add icon conventions |
| 06 Add theme configuration | 14 Document design system rules |
| 07 Add light theme | 15 Refine accessibility defaults |
| 08 Add dark theme if already supported | |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 05

Reusable frontend components

18 commit slots - Complete in order unless the current repository state makes a different order safer.

01 Add Button component

02 Add Input component

03 Add Select component

04 Add Textarea component

05 Add Checkbox component

06 Add Dialog component

07 Add Dropdown component

08 Add Tabs component

09 Add Badge component

10 Add Card component

11 Add Table component

12 Add Pagination component

13 Add Skeleton component

14 Add EmptyState component

15 Add ErrorState component

16 Add Toast or notification primitive

17 Add component export index

18 Validate reusable component imports

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 06

Authentication frontend

17 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|-------------------------------------|--------------------------------------|
| 01 Add auth feature structure | 10 Add auth loading state |
| 02 Add login page | 11 Add invalid credentials state |
| 03 Add registration page if present | 12 Add session expiry handling |
| 04 Add auth form schema | 13 Add organization context handling |
| 05 Add auth API service | 14 Add role-aware navigation |
| 06 Add auth store | 15 Add auth persistence |
| 07 Add session bootstrap | 16 Connect authentication to router |
| 08 Add protected route guard | 17 Validate auth flow end to end |
| 09 Add logout flow | |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 07

Cases frontend

18 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|--------------------------------|------------------------------------|
| 01 Add cases feature structure | 10 Add case sorting |
| 02 Add cases list page | 11 Add case pagination |
| 03 Add case card or row | 12 Add case create flow if present |
| 04 Add case details page | 13 Add case edit flow if present |
| 05 Add case summary panel | 14 Add case activity section |
| 06 Add case status display | 15 Add case empty state |
| 07 Add case metadata display | 16 Add case error state |
| 08 Add case search | 17 Connect cases API |
| 09 Add case filters | 18 Validate cases navigation |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 08

Documents and evidence frontend

20 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|-------------------------------------|---|
| 01 Add documents feature structure | 11 Add evidence list |
| 02 Add document list | 12 Add evidence detail view |
| 03 Add document detail view | 13 Add evidence metadata |
| 04 Add document upload surface | 14 Add evidence linking UI |
| 05 Add upload progress state | 15 Add document and evidence API services |
| 06 Add document metadata panel | 16 Connect document states |
| 07 Add document preview integration | 17 Connect evidence states |
| 08 Add document filtering | 18 Validate document and evidence flows |
| 09 Add document search | 19 Refine upload error handling |
| 10 Add evidence feature structure | 20 Run frontend build |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 09

Hearings, timeline, clients and collaboration UI

18 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|-----------------------------------|--|
| 01 Add hearings feature structure | 10 Add clients feature structure |
| 02 Add hearing list | 11 Add client list |
| 03 Add hearing details | 12 Add client details |
| 04 Add hearing date display | 13 Add collaboration feature structure |
| 05 Add hearing status display | 14 Add collaboration activity panel |
| 06 Add timeline feature structure | 15 Connect hearing API |
| 07 Add timeline view | 16 Connect timeline API |
| 08 Add timeline event component | 17 Connect client API |
| 09 Add timeline filters | 18 Connect collaboration API |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 10

Research, analytics, reports and notifications UI

19 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|------------------------------------|--|
| 01 Add research feature structure | 11 Add report detail |
| 02 Add research search view | 12 Add notifications feature structure |
| 03 Add research result card | 13 Add notification list |
| 04 Add citation display | 14 Add notification state handling |
| 05 Add verification status display | 15 Connect research API |
| 06 Add analytics feature structure | 16 Connect analytics API |
| 07 Add dashboard metrics | 17 Connect reports API |
| 08 Add chart wrappers | 18 Connect notifications API |
| 09 Add reports feature structure | 19 Run frontend integration check |
| 10 Add report list | |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 11

Backend application foundation

17 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|---|--|
| 01 Normalize FastAPI entry point | 10 Add exception structure |
| 02 Add application factory if appropriate | 11 Add health endpoint |
| 03 Add core config module | 12 Add request ID middleware |
| 04 Add environment loading | 13 Add logging setup |
| 05 Add application lifecycle | 14 Add structured logger configuration |
| 06 Add API root router | 15 Normalize backend package exports |
| 07 Add API v1 router | 16 Add backend README |
| 08 Add dependency injection structure | 17 Validate FastAPI startup |
| 09 Add middleware structure | |

Day 11 is the backend baseline. Keep framework configuration separate from domain implementation so later commits remain easy to review.

Day 12

Database, security and migrations

18 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|------------------------------------|--|
| 01 Normalize database package | 10 Add pgvector configuration if present |
| 02 Add async SQLAlchemy engine | 11 Normalize security package |
| 03 Add async session factory | 12 Add password hashing utility |
| 04 Add declarative base | 13 Add JWT utility |
| 05 Normalize model imports | 14 Add authentication dependencies |
| 06 Add repository conventions | 15 Add authorization dependency helpers |
| 07 Normalize Alembic configuration | 16 Add CORS configuration |
| 08 Verify migration discovery | 17 Validate database import chain |
| 09 Add PostgreSQL settings | 18 Run migration checks |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 13

Users, organizations, roles and admin backend

19 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|---------------------------------------|---------------------------------------|
| 01 Add users module structure | 11 Add organization router |
| 02 Normalize user models | 12 Add roles module structure |
| 03 Add user schemas | 13 Normalize role models |
| 04 Add user repository | 14 Add role schemas |
| 05 Add user service | 15 Add role service |
| 06 Add user router | 16 Add role router |
| 07 Add organizations module structure | 17 Add admin module structure |
| 08 Normalize organization models | 18 Add admin authorization rules |
| 09 Add organization schemas | 19 Validate identity and access flows |
| 10 Add organization service | |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 14

Cases, hearings, timeline and evidence backend

20 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|----------------------------------|------------------------------|
| 01 Add cases module structure | 11 Add hearing router |
| 02 Normalize case models | 12 Normalize timeline module |
| 03 Add case schemas | 13 Add timeline schemas |
| 04 Add case repository | 14 Add timeline service |
| 05 Add case service | 15 Add timeline router |
| 06 Add case router | 16 Normalize evidence module |
| 07 Add hearings module structure | 17 Add evidence schemas |
| 08 Normalize hearing models | 18 Add evidence service |
| 09 Add hearing schemas | 19 Add evidence router |
| 10 Add hearing service | 20 Validate core case APIs |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 15

Documents, OCR, drafting and storage backend

20 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|--|---|
| 01 Add documents module structure | 11 Add OCR service structure |
| 02 Normalize document models | 12 Add OCR engine adapter |
| 03 Add document schemas | 13 Add OCR processing service |
| 04 Add document repository | 14 Add document extraction utilities |
| 05 Add document service | 15 Add PDF processing integration |
| 06 Add document router | 16 Add DOCX processing integration |
| 07 Add storage service structure | 17 Add drafting module structure |
| 08 Normalize storage provider interface | 18 Add drafting service |
| 09 Add local storage adapter if present | 19 Add drafting router |
| 10 Add Supabase storage adapter if present | 20 Validate document processing imports |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 16

Research, retrieval, verification and AI services

20 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|------------------------------------|--------------------------------------|
| 01 Normalize research module | 11 Add verification service |
| 02 Add research schemas | 12 Add verification router |
| 03 Add research service | 13 Normalize AI gateway structure |
| 04 Add research router | 14 Add AI provider interface |
| 05 Normalize retrieval module | 15 Add AI gateway routing |
| 06 Add retrieval service | 16 Add AI health handling |
| 07 Add semantic search integration | 17 Add fallback handling if present |
| 08 Add embedding service interface | 18 Normalize agents module |
| 09 Normalize verification module | 19 Add agent service boundaries |
| 10 Add verification schemas | 20 Validate AI and retrieval imports |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 17

Workers, workflows, tasks, notifications and collaboration backend

20 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|--|-----------------------------------|
| 01 Normalize Celery configuration | 11 Add task retry policy |
| 02 Normalize Redis configuration | 12 Add task error handling |
| 03 Add worker package structure | 13 Normalize notifications module |
| 04 Add document processing task | 14 Add notifications schemas |
| 05 Add OCR task integration | 15 Add notifications service |
| 06 Add embedding task integration | 16 Add notifications router |
| 07 Add analytics task if present | 17 Normalize collaboration module |
| 08 Add notification task handling | 18 Add collaboration service |
| 09 Normalize workflows package | 19 Validate worker imports |
| 10 Add workflow orchestration boundaries | 20 Validate workflow imports |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 18

API integration, error handling and security hardening

19 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|--|--|
| 01 Connect frontend auth service to backend | 11 Add request ID propagation |
| 02 Connect frontend cases service to backend | 12 Review rate limiting implementation |
| 03 Connect frontend hearings service to backend | 13 Review authorization boundaries |
| 04 Connect frontend documents service to backend | 14 Review cross-tenant query boundaries |
| 05 Connect frontend evidence service to backend | 15 Review secret handling |
| 06 Connect frontend research service to backend | 16 Review file upload validation |
| 07 Connect frontend analytics service to backend | 17 Review exception leakage |
| 08 Connect frontend reports service to backend | 18 Run frontend and backend smoke checks |
| 09 Normalize API error responses | 19 Run end-to-end API startup validation |
| 10 Add frontend API error mapping | |

Day 18 is integration and hardening. This is where frontend services and backend endpoints meet; keep each connection or hardening concern isolated.

Day 19

Testing, quality gates and CI

20 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|--------------------------------------|-----------------------------------|
| 01 Normalize backend test structure | 11 Add auth UI tests |
| 02 Add backend unit test conventions | 12 Add cases UI tests |
| 03 Add auth tests | 13 Add document UI tests |
| 04 Add case service tests | 14 Add research UI tests |
| 05 Add document service tests | 15 Add integration test structure |
| 06 Add OCR tests | 16 Add AI service mocks |
| 07 Add retrieval tests | 17 Add GitHub Actions workflow |
| 08 Add verification tests | 18 Add backend validation job |
| 09 Normalize frontend test structure | 19 Add frontend validation job |
| 10 Add component test conventions | 20 Run complete test suite |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 20

Infrastructure, deployment and documentation

20 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|---|--|
| 01 Normalize Dockerfile | 11 Add database setup documentation |
| 02 Normalize docker-compose configuration | 12 Add API documentation guide |
| 03 Separate service definitions | 13 Add architecture documentation |
| 04 Configure API container | 14 Add environment configuration guide |
| 05 Configure database container | 15 Add troubleshooting guide |
| 06 Configure Redis container | 16 Add contribution guide |
| 07 Configure worker container | 17 Refine root README |
| 08 Normalize Nginx configuration | 18 Refine PROJECT_STRUCTURE.md |
| 09 Add deployment documentation | 19 Validate Docker configuration |
| 10 Add development setup documentation | 20 Validate documentation links |

Keep the scope narrow. A commit should represent one coherent change that could be understood and reviewed on its own.

Day 21

Refactor, polish, final validation and release preparation

20 commit slots - Complete in order unless the current repository state makes a different order safer.

- | | |
|---------------------------------|---------------------------------------|
| 01 Remove unused imports | 11 Review accessibility labels |
| 02 Remove dead frontend code | 12 Review API route organization |
| 03 Remove dead backend code | 13 Review database model organization |
| 04 Normalize naming conventions | 14 Review worker boundaries |
| 05 Normalize module exports | 15 Review AI service boundaries |
| 06 Reduce duplicate utilities | 16 Run frontend production build |
| 07 Refine error messages | 17 Run backend startup validation |
| 08 Refine loading states | 18 Run test suite |
| 09 Refine empty states | 19 Run repository secret scan |
| 10 Refine responsive behavior | 20 Final repository cleanup |

Final day: only polish and validation. Do not invent a new feature just to increase the commit count; the goal is a stable, understandable repository.

Daily Antigravity Operating Prompt

Save this page inside the repository as a reference. Each day, tell Antigravity only the current day number and let it use the plan above.

You are working on the JurisPulse repository. A PDF named "JurisPulse_21_Day_Git_Upload_Plan.pdf" is present in the repository root.

The developer has told you: "Day X is today."

1. Open the PDF and locate Day X.
2. Inspect the current repository before changing anything.
3. Identify the files that actually belong to today's theme.
4. Implement only real, useful changes from today's commit slots.
5. Keep each commit small and coherent. Prefer one logical change per commit.
6. Use concise engineering commit messages that describe the actual change, for example "Add case list page" or "Normalize database session".
7. Do not create empty commits, placeholder commits, fake timestamps, fabricated issue references, or changes that exist only to increase the number of commits.
8. Do not describe the work as AI-generated or add artificial "AI" language to file descriptions, comments, commit messages, or documentation. Write normal project documentation based on the actual code.
9. Do not perform future-day work unless a dependency must be repaired to keep today's work functional.
10. After each logical change, verify imports/build/tests as appropriate, then create that commit.
11. Before finishing the day, run the relevant validation, check git status, and summarize which commit slots were completed and which were skipped because the corresponding functionality did not exist.
12. Never claim a test or build passed unless you actually ran it.

Suggested human workflow

At the start of a day, say only: Day 07 is today. After Antigravity finishes, review the changed files and the commit list before pushing. The PDF intentionally uses day numbers rather than calendar dates, so the schedule can begin whenever the repository is ready.

Repository hygiene before every push

Check git status; confirm no secrets, .env files, build output, caches, or unrelated files are staged. Confirm the commit history is readable and the working tree is in the expected state. Keep the Git history aligned with actual development work.

Completion target

Across 21 day-plans, the roadmap contains 388 commit slots, with every day between 15 and 20 slots. The sequence moves from repository foundation through frontend, backend, data, AI/document services, workers, integration, testing, infrastructure, and final validation.

What success looks like

By the end, the repository should have a clean structure, understandable commit history, validated builds/tests to the extent supported by the local environment, and documentation that matches the actual implementation. The number of commits is secondary to whether each commit corresponds to a genuine, coherent change.